

Sistema de Monitoreo de Temperatura, Humedad y ACC

Isabella Montoya Díaz, Juan David Tamayo García, Paula Torres Uribe
Electrónica digital II

Índice

1. Identificación del proyecto	2
2. Resumen	2
3. Descripción del hardware	2
4. Descripción del software	3
5. Estructura del código	4
6. Explicación de funciones principales	4
7. Comunicación	4
8. Interfaz de usuario	5
9. Procedimiento de prueba	5
10. Manejo de errores y seguridad	5

1. Identificación del proyecto

- **Nombre del proyecto:** Sistema de Monitoreo Térmico con ESP32 y MicroPython
- **Integrantes del equipo:** Isabella Montoya Díaz, Juan David Tamayo García, Paula Torres uribe

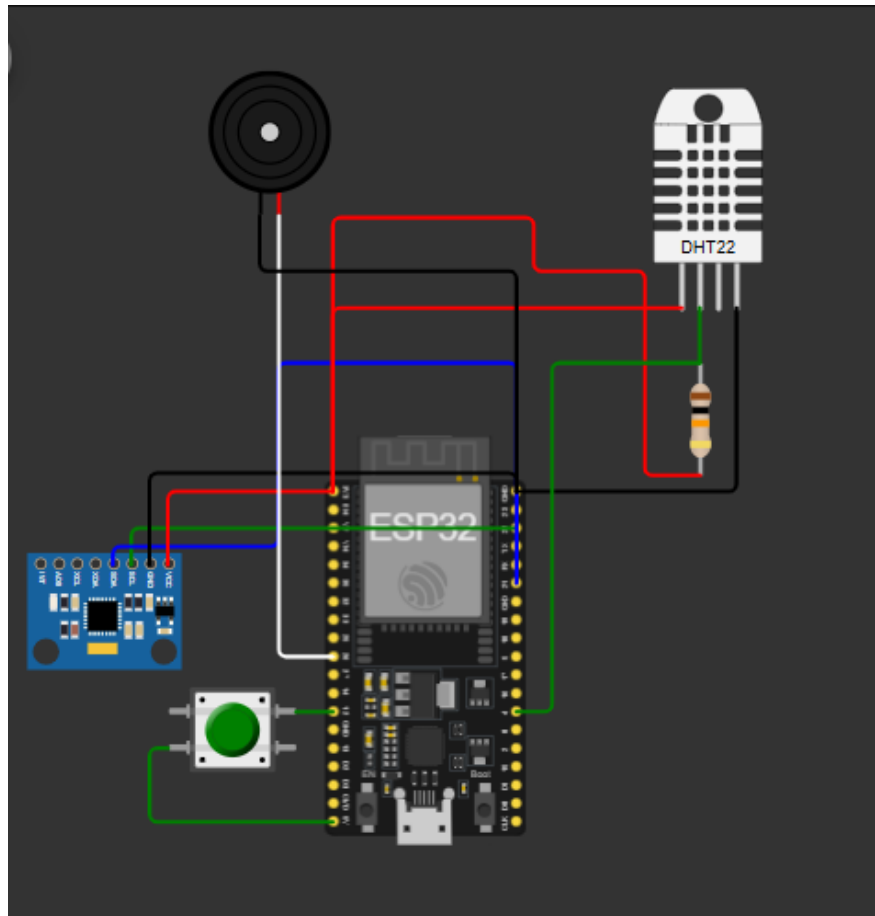
2. Resumen

Este proyecto desarrolla un monitoreo del ambiente en espacios biomédicos mediante un sensor de temperatura y humedad, en conjunto con un acelerómetro que detecta cambios bruscos en la orientación o movimiento del dispositivo. El sistema usa una ESP32, y los datos obtenidos de ambos sensores se visualizan en un servidor web y un chatbot de telegram, el cual envía alertas cuando detecta cambios que no cumplen las condiciones establecidas.

3. Descripción del hardware

- **ESP32** (modelo con ADC y pines GPIO)
- **Sensor DHT11** conectado al pin 4
- **Sensor MPU6050 SCL** conectado al pin 22
- **Sensor MPU6050 SDA** conectado al pin 21
- **Buzzer** conectado al pin 26
- **Pulsador** conectado al pin 12

Diagrama de conexión:



4. Descripción del software

- **Lenguaje usado:** MicroPython ,javascript.
- **Librerías:** machine, time,dht,MPU6050,
utetegram,network,_thread,urequests,flask,collections
- **IDE:** Thonny , Visual Studio Code

Flujo general del programa:

1. Inicia el servidor.
2. Conexión a Wifi.
3. Conexión con el Chat Bot.
4. Conexión con el servidor.
5. Lectura de los sensores .
6. Envío de datos al servidor.

7. Revision de umbrales de alarma .
8. Si se cumple algun umbrual, envio de mensaje de alerta, buzzer y ventana de alerta en el servidor.
9. Lectura de boton de panico , en caso de estar activo muestra alerta en el servidor, chat bot y suena el buzzer.
10. Lectura de mensajes y envio de datos requeridos.

5. Estructura del código

- Archivo principal: `main.py` , `servidor.py`
- se usaron dos mudulos adicionales los cuales fueron: `MPU650.py` , `utelegram.py` .
- Código lineal dentro de un ciclo infinito con lectura y decisión en tiempo real.

6. Explicación de funciones principales

- `handle_message(update)`: Procesa los mensajes que ingresan al telegram.
- `enviar_al_servidor(temp,hum,gforce,boton_panico)`: Envia los datos de los sensores al servidor.
- `recibir_mensajes_telegram()`: Recibe los mensajes
- `activar_buzzer(frecuencia)`: Activa el buzzer a una frecuencia especifica.
- `desactivar_buzzer()`: Desactiva el buzzer.
- `home()`: Muestra la pagina web con el panel de monitoreo.
- `api_last()`: Entrega al navegador los ultimos datos para que se actualice.
- `ingest()`: Revisa los datos que envia la esp32 y revisa quien los envia y si el dispositivo que los envia tiene la contraseña correcta.

7. Comunicación

Este proyecto utiliza comunicación inalámbrica basado en HTTP para enviar datos al servidor web, ademas se uso comunicacion serial i2C.

8. Interfaz de usuario

Se incluye un sistema de visualización en un servidor web el cuál tiene un panel donde muestra los valores normalizados, los cambios de cada valor, las respectivas graficas y las alertas ,ademas un chatbot de telegram que envía alertas automáticas cuando se superan ciertos umbrales establecidos previamente en la código y envia datos que se soliciten.

9. Procedimiento de prueba

1. Conectar el ESP32 al computador.
2. Iniciar el servidor en visual.
3. Conectar el ESP32 a wifi.
4. Conectar con el chat bot de telegram.
5. Modificar los parametros (Temperatura, Humedad, Gforce) y observar los cambios mediante el servidor y el chat_bot.
6. Al llegar algun umbral evidenciar las diferentes alarmas para cada parametro, tanto en el servidor como en el chat bot de telegram, ademas de los diferentes sonidos de alarma del buzzer.
7. Solicitar al mediante el chat bot de telegram diferentes datos de los parametros.
8. Precionar el boton de panico para evidenciar las alarmas.

10. Manejo de errores y seguridad

- Se asume que el sensor DHT11 puede fallar en la lectura; se recomienda agregar `try-except` en implementaciones reales.
- Revisar que todos los dispositivos esten conectados a la misma red wifi.
- Que esta actualizado el api del bot y el id del usuario.